Student Management System

Generated by Doxygen 1.13.2

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Class Documentation

4.1 Laikas Class Reference

Class for measuring execution time.

`#include <laikas.h>`

Public Member Functions

- Laikas (const std::string &pavadinimas)
  Constructor.
- void pradeti ()
  Start the timer Records the starting time point.
- void baigti ()
  Stop the timer and print the elapsed time Records the ending time point and outputs the time difference.
- double gautiLaikoSkirtuma ()
  Get the elapsed time in seconds.

Private Attributes

- std::chrono::high_resolution_clock::time_point start
  Start time point.
- std::chrono::high_resolution_clock::time_point end
  End time point.
- std::string veiksmoPavadinimas
  Name of the operation being timed.

4.1.1 Detailed Description

Class for measuring execution time.

This class provides functionality to measure and report the execution time of code segments using high-resolution clock.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Laikas()

```
Laikas::Laikas (
            const std::string & pavadinimas)
```

Constructor.

Parameters

pavadinimas	Name of the operation to time

4.1.3 Member Function Documentation

4.1.3.1 baigti()

```
void Laikas::baigti ()
```

Stop the timer and print the elapsed time Records the ending time point and outputs the time difference.

4.1.3.2 gautiLaikoSkirtuma()

```
double Laikas::gautiLaikoSkirtuma ()
```

Get the elapsed time in seconds.

Returns

Time difference between start and end in seconds

4.1.3.3 pradeti()

```
void Laikas::pradeti ()
```

Start the timer Records the starting time point.

4.1.4 Member Data Documentation

4.1.4.1 end

```
std::chrono::high_resolution_clock::time_point Laikas::end  [private]
```

End time point.

4.1.4.2 start

```
std::chrono::high_resolution_clock::time_point Laikas::start  [private]
```

Start time point.

4.1.4.3 veiksmoPavadinimas

```
std::string Laikas::veiksmoPavadinimas  [private]
```

Name of the operation being timed.

The documentation for this class was generated from the following files:

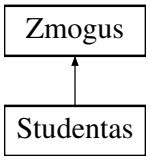
- laikas.h
- laikas.cpp

4.2 Studentas Class Reference

Student class that inherits from Zmogus (Person) abstract class.

```
#include <funkcijos.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- Studentas ()
  Default constructor Initializes a student with default values.
- Studentas (std::istream &is)
  Constructor that initializes a student from input stream.
- Studentas (const std::string &vardas, const std::string &pavarde)
  Constructor with name and surname.
- Studentas (const Studentas &other)
  Copy constructor.
- Studentas & operator= (const Studentas &other)
  Copy assignment operator.
- Studentas (Studentas &&other) noexcept
  Move constructor.
- Studentas & operator= (Studentas &&other) noexcept
  Move assignment operator.
- ~Studentas () override
  Virtual destructor Overrides the pure virtual destructor from Zmogus.
- void print (std::ostream &os) const override
  Prints student information to output stream.
- void read (std::istream &is) override
  Reads student information from input stream.
- std::vector< int > nd () const
  Get homework grades.

- int egzaminas () const

  Get exam grade.
- double galutinis () const

  Get final grade.
- void setEgzaminas (int egzaminas)

  Set exam grade.
- void setGalutinis (double galutinis)

  Set final grade.
- std::istream & readStudent (std::istream &is)

  Read student data from input stream.
- void addND (int pazymys)

  Add a homework grade.
- void clearND ()

  Clear all homework grades.
- double galBalas () const

  Calculate final grade (average method)
- double galBalasMed () const

  Calculate final grade (median method)

Public Member Functions inherited from Zmogus

- Zmogus ()=default

  Default constructor.
- Zmogus (const std::string &vardas, const std::string &pavarde)

  Constructor with name and surname.
- Zmogus (const Zmogus &other)

  Copy constructor.
- Zmogus & operator= (const Zmogus &other)

  Copy assignment operator.
- Zmogus (Zmogus &&other) noexcept

  Move constructor.
- Zmogus & operator= (Zmogus &&other) noexcept

  Move assignment operator.
- virtual ~Zmogus ()=0

  Virtual destructor Pure virtual destructor makes this class abstract.
- std::string vardas () const

  Get first name.
- std::string pavarde () const

  Get last name.
- void setVardas (const std::string &vardas)

  Set first name.
- void setPavarde (const std::string &pavarde)

  Set last name.

Static Public Member Functions

- template<typename Container>
  static double skaiciuotiVid (const Container &nd)
  Calculate average of homework grades.
- template<typename Container>
  static double skaiciuotiMed (Container nd)
  Calculate median of homework grades.
- template<typename Container>
  static void nuskaitymasFile (Container &grupe, const std::string &failoPavadinimas)
  Read students from file (static method)

Static Public Attributes

- static int destruktoriuSk = 0
  Static counter for tracking destructor calls (for testing)

Private Attributes

- std::vector< int > nd_
  Vector of homework (namų darbų) scores.
- int egzaminas_
  Exam score.
- double galutinis_
  Final grade.

Additional Inherited Members

Protected Attributes inherited from Zmogus

- std::string vardas_
  First name of the person.
- std::string pavarde_
  Last name of the person.

4.2.1 Detailed Description

Student class that inherits from Zmogus (Person) abstract class.

This class represents a student with homework grades, exam score and final grade. It inherits from the abstract Zmogus class and implements its pure virtual methods.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Studentas() [1/5]

```
Studentas::Studentas ()  [inline]
```

Default constructor Initializes a student with default values.

4.2.2.2 Studentas() [2/5]

```
Studentas::Studentas (
            std::istream & is)
```

Constructor that initializes a student from input stream.

Parameters

is	Input stream to read from

4.2.2.3 Studentas() [3/5]

```
Studentas::Studentas (
            const std::string & vardas,
            const std::string & pavarde)  [inline]
```

Constructor with name and surname.

Parameters

vardas	Student's first name
pavarde	Student's last name

4.2.2.4 Studentas() [4/5]

```
Studentas::Studentas (
            const Studentas & other)
```

Copy constructor.

Parameters

other	Another student to copy from

4.2.2.5 Studentas() [5/5]

```
Studentas::Studentas (
            Studentas && other)  [noexcept]
```

Move constructor.

Parameters

other	Another student to move from

4.2.2.6 ~Studentas()

```
Studentas::~Studentas ()  [override]
```

Virtual destructor Overrides the pure virtual destructor from Zmogus.

4.2.3 Member Function Documentation

4.2.3.1 addND()

```
void Studentas::addND (
            int pazymys)
```

Add a homework grade.

Parameters

pazymys	Grade to add

Exceptions

std::invalid_argument	If grade is not between 1 and 10

4.2.3.2 clearND()

```
void Studentas::clearND ()
```

Clear all homework grades.

4.2.3.3 egzaminas()

```
int Studentas::egzaminas () const  [inline]
```

Get exam grade.

Returns

Exam grade

4.2.3.4 galBalas()

```
double Studentas::galBalas () const  [inline]
```

Calculate final grade (average method)

Returns

Final grade based on homework average and exam

4.2.3.5 galBalasMed()

```
double Studentas::galBalasMed () const  [inline]
```

Calculate final grade (median method)

Returns

Final grade based on homework median and exam

4.2.3.6 galutinis()

```
double Studentas::galutinis () const  [inline]
```

Get final grade.

Returns

Final grade

4.2.3.7 nd()

```
std::vector< int > Studentas::nd () const  [inline]
```

Get homework grades.

Returns

Vector of homework grades

4.2.3.8 nuskaitymasFile()

```
template<typename Container>
static void Studentas::nuskaitymasFile (
            Container & grupe,
            const std::string & failoPavadinimas)  [inline], [static]
```

Read students from file (static method)

Parameters

grupe	Container to store students
failoPavadinimas	Name of the file to read from

4.2.3.9 operator=() [1/2]

```
Studentas & Studentas::operator= (
            const Studentas & other)
```

Copy assignment operator.

Parameters

other	Another student to copy from

Returns

Reference to this student after assignment

4.2.3.10 operator=() [2/2]

```
Studentas & Studentas::operator= (
            Studentas && other)  [noexcept]
```

Move assignment operator.

Parameters

other	Another student to move from

Returns

Reference to this student after assignment

4.2.3.11 print()

```
void Studentas::print (
            std::ostream & os) const  [override], [virtual]
```

Prints student information to output stream.

Parameters

os	Output stream to print to

Implements Zmogus.

4.2.3.12 read()

```
void Studentas::read (
            std::istream & is)  [override], [virtual]
```

Reads student information from input stream.

Parameters

is	Input stream to read from

Implements Zmogus.

4.2.3.13 readStudent()

```
std::istream & Studentas::readStudent (
            std::istream & is)
```

Read student data from input stream.

Parameters

is	Input stream to read from

Returns

Reference to the input stream

4.2.3.14 setEgzaminas()

```
void Studentas::setEgzaminas (
            int egzaminas)  [inline]
```

Set exam grade.

Parameters

egzaminas	Exam grade to set

4.2.3.15 setGalutinis()

```
void Studentas::setGalutinis (
            double galutinis)  [inline]
```

Set final grade.

Parameters

galutinis	Final grade to set

4.2.3.16 skaiciuotiMed()

```
template<typename Container>
static double Studentas::skaiciuotiMed (
            Container nd)  [inline], [static]
```

Calculate median of homework grades.

Parameters

nd	Container of grades (will be sorted)

Returns

Median value

4.2.3.17 skaiciuotiVid()

```
template<typename Container>
static double Studentas::skaiciuotiVid (
            const Container & nd)  [inline], [static]
```

Calculate average of homework grades.

Parameters

nd	Container of grades

Returns

Average value

4.2.4 Member Data Documentation

4.2.4.1 destruktoriuSk

```
int Studentas::destruktoriuSk = 0  [static]
```

Static counter for tracking destructor calls (for testing)

4.2.4.2 egzaminas_

```
int Studentas::egzaminas_  [private]
```

Exam score.

4.2.4.3 galutinis_

```
double Studentas::galutinis_  [private]
```

Final grade.

4.2.4.4 nd_

```
std::vector<int> Studentas::nd_  [private]
```

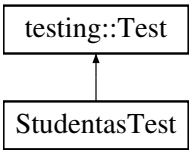
Vector of homework (namų darbų) scores.

The documentation for this class was generated from the following files:

- funkcijos.h
- funkcijos.cpp

4.3 StudentasTest Class Reference

Inheritance diagram for StudentasTest:



Protected Member Functions

- void SetUp () override

Protected Attributes

- Studentas testStudent

4.3.1 Member Function Documentation

4.3.1.1 SetUp()

```
void StudentasTest::SetUp ()  [inline], [override], [protected]
```

4.3.2 Member Data Documentation

4.3.2.1 testStudent

```
Studentas StudentasTest::testStudent  [protected]
```

The documentation for this class was generated from the following file:

- testai.cpp

4.4 Vector< T > Class Template Reference

Dinaminis šabloninis konteineris, panašus į std::vector.

```
#include <vector.h>
```

Public Types

- using value_type = T
  Elemento tipas.
- using size_type = std::size_t
  Tipas dydžio žymėjimui.
- using difference_type = std::ptrdiff_t
  Skirtumo tipas tarp iteratorių
- using reference = T&
  Nuoroda į elementą
- using const_reference = const T&
  Konstanta nuoroda į elementą
- using pointer = T*
  Rodyklė į elementą
- using const_pointer = const T*
  Konstanta rodyklė į elementą
- using iterator = T*
  Iteratorius.
- using const_iterator = const T*
  Konstanta iteratorius.
- using reverse_iterator = std::reverse_iterator<iterator>
  Atvirkštinis iteratorius.
- using const_reverse_iterator = std::reverse_iterator<const_iterator>
  Konstanta atvirkštinis iteratorius.

Public Member Functions

- Vector ()=default

  Numatytasis konstruktorius.
- Vector (size_type count, const T &value=T())

  Konstruktorius su elementų skaičiumi ir pradine verte.
- Vector (std::initializer_list< T > init)

  Konstruktorius iš initializer_list.
- Vector (const Vector &other)

  Kopijavimo konstruktorius.
- Vector (Vector &&other) noexcept

  Perkėlimo konstruktorius.
- ~Vector ()

  Destruktorius.
- Vector & operator= (const Vector &other)
- Vector & operator= (Vector &&other) noexcept
- Vector & operator= (std::initializer_list< T > ilist)
- reference at (size_type pos)

  Prieiga su ribų tikrinimu.
- const_reference at (size_type pos) const

  Prieiga su ribų tikrinimu (konstanta).
- reference operator[ ] (size_type pos)
- const_reference operator[ ] (size_type pos) const
- reference front ()
- const_reference front () const
- reference back ()
- const_reference back () const
- pointer data () noexcept
- const_pointer data () const noexcept
- iterator begin () noexcept
- const_iterator begin () const noexcept
- const_iterator cbegin () const noexcept
- iterator end () noexcept
- const_iterator end () const noexcept
- const_iterator cend () const noexcept
- reverse_iterator rbegin () noexcept
- const_reverse_iterator rbegin () const noexcept
- const_reverse_iterator crbegin () const noexcept
- reverse_iterator rend () noexcept
- const_reverse_iterator rend () const noexcept
- const_reverse_iterator crend () const noexcept
- bool empty () const noexcept
- size_type size () const noexcept
- size_type capacity () const noexcept
- void reserve (size_type new_cap)
- void shrink_to_fit ()
- void clear () noexcept
- iterator insert (const_iterator pos, const T &value)
- iterator insert (const_iterator pos, T &&value)
- iterator insert (const_iterator pos, size_type count, const T &value)
- iterator erase (const_iterator pos)
- iterator erase (const_iterator first, const_iterator last)
- void push_back (const T &value)

- void push_back (T &&value)
- template<class... Args>
  reference emplace_back (Args &&... args)

  Sukuria objektą gale, panaudojant nurodytus argumentus.
- void pop_back ()
- void resize (size_type count)
- void resize (size_type count, const T &value)
- void swap (Vector &other) noexcept
- void assign (size_type count, const T &value)
- template<class InputIt>
  void assign (InputIt first, InputIt last)
- void assign (std::initializer_list< T > ilist)
- size_t reallocations () const

  Gauk kiek kartų buvo atlikta atminties perskirstymų.

Private Member Functions

- void reallocate (size_type new_capacity)

  Atminties perskirstymas naujai talpai.
- void destroy_elements ()

  Naikina visus elementus.

Private Attributes

- pointer data_ = nullptr

  Rodyklė į duomenų masyvą
- size_type size_ = 0

  Dabartinis elementų skaičius.
- size_type capacity_ = 0

  Dabartinė talpa (alokacijos dydis)
- size_t realloc_count_ = 0

  Kiek kartų buvo perskirstyta atmintis.

Friends

- bool operator== (const Vector &lhs, const Vector &rhs)
- bool operator!= (const Vector &lhs, const Vector &rhs)

4.4.1 Detailed Description

template<typename T>
class Vector< T >

Dinaminis šabloninis konteineris, panašus į std::vector.

Template Parameters

T	Elementų tipas.

4.4.2 Member Typedef Documentation

4.4.2.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const T*
```

Konstanta iteratorius.

4.4.2.2 const_pointer

```
template<typename T>
using Vector< T >::const_pointer = const T*
```

Konstanta rodyklė į elementą

4.4.2.3 const_reference

```
template<typename T>
using Vector< T >::const_reference = const T&
```

Konstanta nuoroda į elementą

4.4.2.4 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Konstanta atvirkštinis iteratorius.

4.4.2.5 difference_type

```
template<typename T>
using Vector< T >::difference_type = std::ptrdiff_t
```

Skirtumo tipas tarp iteratorių

4.4.2.6 iterator

```
template<typename T>
using Vector< T >::iterator = T*
```

Iteratorius.

4.4.2.7 pointer

```
template<typename T>
using Vector< T >::pointer = T*
```

Rodyklė į elementą

4.4.2.8 reference

```
template<typename T>
using Vector< T >::reference = T&
```

Nuoroda į elementą

4.4.2.9 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

Atvirkštinis iteratorius.

4.4.2.10 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

Tipas dydžio žymėjimui.

4.4.2.11 value_type

```
template<typename T>
using Vector< T >::value_type = T
```

Elemento tipas.

4.4.3 Constructor & Destructor Documentation

4.4.3.1 Vector() [1/5]

```
template<typename T>
Vector< T >::Vector ()  [default]
```

Numatytasis konstruktorius.

4.4.3.2 Vector() [2/5]

```
template<typename T>
Vector< T >::Vector (
            size_type count,
            const T & value = T())  [inline], [explicit]
```

Konstruktorius su elementų skaičiumi ir pradine verte.

Parameters

count	Elementų kiekis.
value	Pradinė reikšmė kiekvienam elementui.

4.4.3.3 Vector() [3/5]

```
template<typename T>
Vector< T >::Vector (
            std::initializer_list< T > init)  [inline]
```

Konstruktorius iš initializer_list.

Parameters

init	Inicilizacijos sąrašas.

4.4.3.4 Vector() [4/5]

```
template<typename T>
Vector< T >::Vector (
            const Vector< T > & other)  [inline]
```

Kopijavimo konstruktorius.

Parameters

other	Kitas vektorius.

4.4.3.5 Vector() [5/5]

```
template<typename T>
Vector< T >::Vector (
            Vector< T > && other)  [inline], [noexcept]
```

Perkėlimo konstruktorius.

Parameters

other	Perkeliamas vektorius.

4.4.3.6 ~Vector()

```
template<typename T>
Vector< T >::~Vector ()  [inline]
```

Destruktorius.

4.4.4 Member Function Documentation

4.4.4.1 assign() [1/3]

```
template<typename T>
template<class InputIt>
void Vector< T >::assign (
            InputIt first,
            InputIt last)  [inline]
```

4.4.4.2 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
            size_type count,
            const T & value)  [inline]
```

4.4.4.3 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
            std::initializer_list< T > ilist)  [inline]
```

4.4.4.4 at() [1/2]

```
template<typename T>
reference Vector< T >::at (
            size_type pos)  [inline]
```

Prieiga su ribų tikrinimu.

Parameters

pos	Indeksas.

Returns

Nuoroda į elementą.

4.4.4.5 at() [2/2]

```
template<typename T>
const_reference Vector< T >::at (
            size_type pos) const  [inline]
```

Prieiga su ribų tikrinimu (konstanta).

Parameters

pos	Indeksas.

Returns

Konstanta nuoroda į elementą.

4.4.4.6 back() [1/2]

```
template<typename T>
reference Vector< T >::back ()  [inline]
```

4.4.4.7 back() [2/2]

```
template<typename T>
const_reference Vector< T >::back () const  [inline]
```

4.4.4.8 begin() [1/2]

```
template<typename T>
const_iterator Vector< T >::begin () const  [inline], [noexcept]
```

4.4.4.9 begin() [2/2]

```
template<typename T>
iterator Vector< T >::begin ()  [inline], [noexcept]
```

4.4.4.10 capacity()

```
template<typename T>
size_type Vector< T >::capacity () const  [inline], [noexcept]
```

4.4.4.11 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const  [inline], [noexcept]
```

4.4.4.12 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const  [inline], [noexcept]
```

4.4.4.13 clear()

```
template<typename T>
void Vector< T >::clear ()  [inline], [noexcept]
```

4.4.4.14 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const  [inline], [noexcept]
```

4.4.4.15 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const  [inline], [noexcept]
```

4.4.4.16 data() [1/2]

```
template<typename T>
const_pointer Vector< T >::data () const  [inline], [noexcept]
```

4.4.4.17 data() [2/2]

```
template<typename T>
pointer Vector< T >::data ()  [inline], [noexcept]
```

4.4.4.18 destroy_elements()

```
template<typename T>
void Vector< T >::destroy_elements ()  [inline], [private]
```

Naikina visus elementus.

4.4.4.19 emplace_back()

```
template<typename T>
template<class...  Args>
reference Vector< T >::emplace_back (
            Args &&...  args)  [inline]
```

Sukuria objektą gale, panaudojant nurodytus argumentus.

Template Parameters

Args	Argumentų tipai.

Parameters

args	Argumentai, naudojami sukurti objektui.

Returns

Nuoroda į naują objektą.

4.4.4.20 empty()

```
template<typename T>
bool Vector< T >::empty () const  [inline], [noexcept]
```

4.4.4.21 end() [1/2]

```
template<typename T>
const_iterator Vector< T >::end () const  [inline], [noexcept]
```

4.4.4.22 end() [2/2]

```
template<typename T>
iterator Vector< T >::end ()  [inline], [noexcept]
```

4.4.4.23 erase() [1/2]

```
template<typename T>
iterator Vector< T >::erase (
            const_iterator first,
            const_iterator last)  [inline]
```

4.4.4.24 erase() [2/2]

```
template<typename T>
iterator Vector< T >::erase (
            const_iterator pos)  [inline]
```

4.4.4.25 front() [1/2]

```
template<typename T>
reference Vector< T >::front ()  [inline]
```

4.4.4.26 front() [2/2]

```
template<typename T>
const_reference Vector< T >::front () const  [inline]
```

4.4.4.27 insert() [1/3]

```
template<typename T>
iterator Vector< T >::insert (
            const_iterator pos,
            const T & value)  [inline]
```

4.4.4.28 insert() [2/3]

```
template<typename T>
iterator Vector< T >::insert (
            const_iterator pos,
            size_type count,
            const T & value)  [inline]
```

4.4.4.29 insert() [3/3]

```
template<typename T>
iterator Vector< T >::insert (
            const_iterator pos,
            T && value)  [inline]
```

4.4.4.30 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
            const Vector< T > & other)  [inline]
```

4.4.4.31 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
            std::initializer_list< T > ilist)  [inline]
```

4.4.4.32 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
            Vector< T > && other)  [inline], [noexcept]
```

4.4.4.33 operator[]() [1/2]

```
template<typename T>
reference Vector< T >::operator[] (
            size_type pos)  [inline]
```

4.4.4.34 operator[]() [2/2]

```
template<typename T>
const_reference Vector< T >::operator[] (
            size_type pos) const  [inline]
```

4.4.4.35 pop_back()

```
template<typename T>
void Vector< T >::pop_back ()  [inline]
```

4.4.4.36 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
            const T & value)  [inline]
```

4.4.4.37 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
            T && value)  [inline]
```

4.4.4.38 rbegin() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const  [inline], [noexcept]
```

4.4.4.39 rbegin() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin ()  [inline], [noexcept]
```

4.4.4.40 reallocate()

```
template<typename T>
void Vector< T >::reallocate (
            size_type new_capacity)  [inline], [private]
```

Atminties perskirstymas naujai talpai.

Parameters

new_capacity	Nauja talpa.

4.4.4.41 reallocations()

```
template<typename T>
size_t Vector< T >::reallocations () const  [inline]
```

Gauk kiek kartų buvo atlikta atminties perskirstymų.

Returns

Perskirstymų skaičius.

4.4.4.42 rend() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const  [inline], [noexcept]
```

4.4.4.43 rend() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rend ()  [inline], [noexcept]
```

4.4.4.44 reserve()

```
template<typename T>
void Vector< T >::reserve (
            size_type new_cap)  [inline]
```

4.4.4.45 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
            size_type count)  [inline]
```

4.4.4.46 resize() [2/2]

```
template<typename T>
void Vector< T >::resize (
            size_type count,
            const T & value)  [inline]
```

4.4.4.47 shrink_to_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit ()  [inline]
```

4.4.4.48 size()

```
template<typename T>
size_type Vector< T >::size () const  [inline], [noexcept]
```

4.4.4.49 swap()

```
template<typename T>
void Vector< T >::swap (
            Vector< T > & other)  [inline], [noexcept]
```

4.4.5 Friends And Related Symbol Documentation

4.4.5.1 operator"!=

```
template<typename T>
bool operator!= (
            const Vector< T > & lhs,
            const Vector< T > & rhs)  [friend]
```

4.4.5.2 operator==

```
template<typename T>
bool operator== (
            const Vector< T > & lhs,
            const Vector< T > & rhs)  [friend]
```

4.4.6 Member Data Documentation

4.4.6.1 capacity_

```
template<typename T>
size_type Vector< T >::capacity_ = 0  [private]
```

Dabartinė talpa (alokacijos dydis)

4.4.6.2 data_

```
template<typename T>
pointer Vector< T >::data_ = nullptr  [private]
```

Rodyklė į duomenų masyvą

4.4.6.3 realloc_count_

```
template<typename T>
size_t Vector< T >::realloc_count_ = 0  [private]
```

Kiek kartų buvo perskirstyta atmintis.

4.4.6.4 size_

```
template<typename T>
size_type Vector< T >::size_ = 0  [private]
```

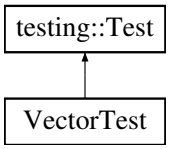
Dabartinis elementų skaičius.

The documentation for this class was generated from the following file:

- vector.h

4.5 VectorTest Class Reference

Inheritance diagram for VectorTest:



Protected Member Functions

- void SetUp () override
- void TearDown () override
- template<typename T>
  void expectEqual (const Vector< T > &v1, const std::vector< T > &v2)

4.5.1 Member Function Documentation

4.5.1.1 expectEqual()

```
template<typename T>
void VectorTest::expectEqual (
            const Vector< T > & v1,
            const std::vector< T > & v2)  [inline], [protected]
```

4.5.1.2 SetUp()

```
void VectorTest::SetUp ()  [inline], [override], [protected]
```

4.5.1.3 TearDown()

```
void VectorTest::TearDown ()  [inline], [override], [protected]
```

The documentation for this class was generated from the following file:

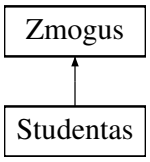
- vector_testai.cpp

4.6 Zmogus Class Reference

Abstract base class representing a person.

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- Zmogus ()=default
  Default constructor.
- Zmogus (const std::string &vardas, const std::string &pavarde)
  Constructor with name and surname.
- Zmogus (const Zmogus &other)
  Copy constructor.
- Zmogus & operator= (const Zmogus &other)
  Copy assignment operator.
- Zmogus (Zmogus &&other) noexcept
  Move constructor.
- Zmogus & operator= (Zmogus &&other) noexcept
  Move assignment operator.
- virtual ~Zmogus ()=0
  Virtual destructor Pure virtual destructor makes this class abstract.
- virtual void print (std::ostream &os) const =0
  Print person information to output stream.
- virtual void read (std::istream &is)=0
  Read person information from input stream.
- std::string vardas () const
  Get first name.
- std::string pavarde () const
  Get last name.
- void setVardas (const std::string &vardas)
  Set first name.
- void setPavarde (const std::string &pavarde)
  Set last name.

Protected Attributes

- std::string vardas_

  First name of the person.
- std::string pavarde_

  Last name of the person.

4.6.1 Detailed Description

Abstract base class representing a person.

This class serves as an abstract base class for all person types. It implements common attributes (name, surname) and declares pure virtual methods.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus ()  [default]
```

Default constructor.

4.6.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
            const std::string & vardas,
            const std::string & pavarde)  [inline]
```

Constructor with name and surname.

Parameters

vardas	First name
pavarde	Last name

4.6.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
            const Zmogus & other)  [inline]
```

Copy constructor.

Parameters

other	Another person to copy from

4.6.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
            Zmogus && other)  [noexcept]
```

Move constructor.

Parameters

other	Another person to move from

4.6.2.5 ~Zmogus()

```
Zmogus::~Zmogus ()  [pure virtual]
```

Virtual destructor Pure virtual destructor makes this class abstract.

4.6.3 Member Function Documentation

4.6.3.1 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
            const Zmogus & other)
```

Copy assignment operator.

Parameters

other	Another person to copy from

Returns

Reference to this person after assignment

4.6.3.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
            Zmogus && other)  [noexcept]
```

Move assignment operator.

Parameters

other	Another person to move from

Returns

Reference to this person after assignment

4.6.3.3 pavarde()

```
std::string Zmogus::pavarde () const  [inline]
```

Get last name.

Returns

Last name of the person

4.6.3.4 print()

```
virtual void Zmogus::print (
            std::ostream & os) const  [pure virtual]
```

Print person information to output stream.

Parameters

os	Output stream to print to

Pure virtual method that derived classes must implement

Implemented in Studentas.

4.6.3.5 read()

```
virtual void Zmogus::read (
            std::istream & is)  [pure virtual]
```

Read person information from input stream.

Parameters

is	Input stream to read from

Pure virtual method that derived classes must implement

Implemented in Studentas.

4.6.3.6 setPavarde()

```
void Zmogus::setPavarde (
            const std::string & pavarde)  [inline]
```

Set last name.

Parameters

pavarde	Last name to set

4.6.3.7 setVardas()

```
void Zmogus::setVardas (
            const std::string & vardas)  [inline]
```

Set first name.

Parameters

vardas	First name to set

4.6.3.8 vardas()

```
std::string Zmogus::vardas () const  [inline]
```

Get first name.

Returns

First name of the person

4.6.4 Member Data Documentation

4.6.4.1 pavarde_

```
std::string Zmogus::pavarde_  [protected]
```

Last name of the person.

4.6.4.2 vardas_

```
std::string Zmogus::vardas_  [protected]
```

First name of the person.

The documentation for this class was generated from the following files:

- zmogus.h
- zmogus.cpp

Chapter 5

File Documentation

5.1 funkcijos.cpp File Reference

```
#include "funkcijos.h"
#include "laikas.h"
#include "vector.h"
#include <cstdlib>
#include <ctime>
```

Functions

- bool compareByVardas (const Studentas &a, const Studentas &b)

  Compare students by first name.
- bool compareByPavarde (const Studentas &a, const Studentas &b)

  Compare students by last name.
- bool compareByGalutinis (const Studentas &a, const Studentas &b)

  Compare students by final grade (descending)
- template<typename Container>
  void testuotiDuomenuApdorojimaImpl (const std::string &aplankas, int skaicius)

  Implementation of data processing test.
- void testuotiDuomenuApdorojima (const std::string &aplankas, int skaicius, int testChoice)

  Test data processing performance.
- template void testuotiDuomenuApdorojimaImpl< std::vector< Studentas > > (const std::string &aplankas, int skaicius)
- template void testuotiDuomenuApdorojimaImpl< Vector< Studentas > > (const std::string &aplankas, int skaicius)
- void testuotiStudentoMetodus ()

  Test Studentas class methods.
- void testuotiZmogausKlase ()

  Test Zmogus class.

5.1.1 Function Documentation

5.1.1.1 compareByGalutinis()

```
bool compareByGalutinis (
            const Studentas & a,
            const Studentas & b)
```

Compare students by final grade (descending)

5.1.1.2 compareByPavarde()

```
bool compareByPavarde (
            const Studentas & a,
            const Studentas & b)
```

Compare students by last name.

5.1.1.3 compareByVardas()

```
bool compareByVardas (
            const Studentas & a,
            const Studentas & b)
```

Compare students by first name.

5.1.1.4 testuotiDuomenuApdorojima()

```
void testuotiDuomenuApdorojima (
            const std::string & aplankas,
            int skaicius,
            int testChoice)
```

Test data processing performance.

Parameters

aplankas	Directory for files
skaicius	Number of students
testChoice	Test choice (2 for std::vector, 3 for custom Vector)

5.1.1.5 testuotiDuomenuApdorojimaImpl()

```
template<typename Container>
void testuotiDuomenuApdorojimaImpl (
            const std::string & aplankas,
            int skaicius)
```

Implementation of data processing test.

Parameters

aplankas	Directory for test files
skaicius	Number of students

5.1.1.6 testuotiDuomenuApdorojimaImpl< std::vector< Studentas > >()

```
template void testuotiDuomenuApdorojimaImpl< std::vector< Studentas > > (
            const std::string & aplankas,
            int skaicius)
```

5.1.1.7 testuotiDuomenuApdorojimaImpl< Vector< Studentas > >()

```
template void testuotiDuomenuApdorojimaImpl< Vector< Studentas > > (
            const std::string & aplankas,
            int skaicius)
```

5.1.1.8 testuotiStudentoMetodus()

```
void testuotiStudentoMetodus ()
```

Test Studentas class methods.

5.1.1.9 testuotiZmogausKlase()

```
void testuotiZmogausKlase ()
```

Test Zmogus class.

5.2 funkcijos.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <execution>
#include <numeric>
#include <cassert>
#include "zmogus.h"
```

Classes

- class Studentas

  Student class that inherits from Zmogus (Person) abstract class.

Functions

- bool compareByVardas (const Studentas &a, const Studentas &b)

  Compare students by first name.
- bool compareByPavarde (const Studentas &a, const Studentas &b)

  Compare students by last name.
- bool compareByGalutinis (const Studentas &a, const Studentas &b)

  Compare students by final grade (descending)
- template<typename Container>
  void skaitytiIsFailo (Container &grupe, const std::string &failoPavadinimas)

  Read students from file into container.
- template<typename Container>
  void sortStudentai (Container &grupe, char sortingOption)

  Sort students in container.
- template<typename Container>
  void skirstytiStudentus (Container &grupe, Container &kietiakiai, Container &vargsai)

  Separate students into two groups based on final grade.
- template<typename Container>
  void isvestiStudentusIFaila (const Container &studentai, const std::string &failoPavadinimas, char ats)

  Output students to file.
- template<typename Container>
  void generuotiStudentus (Container &grupe, int kiekis)

  Generate random students for testing.
- template<typename Container>
  void testuotiDuomenuApdorojimaImpl (const std::string &aplankas, int skaicius)

  Implementation of data processing test.
- void testuotiDuomenuApdorojima (const std::string &aplankas, int skaicius, int testChoice)

  Test data processing performance.
- void testuotiStudentoMetodus ()

  Test Studentas class methods.
- void testuotiZmogausKlase ()

  Test Zmogus class.

5.2.1 Function Documentation

5.2.1.1 compareByGalutinis()

```
bool compareByGalutinis (
            const Studentas & a,
            const Studentas & b)
```

Compare students by final grade (descending)

5.2.1.2 compareByPavarde()

```
bool compareByPavarde (
            const Studentas & a,
            const Studentas & b)
```

Compare students by last name.

5.2.1.3 compareByVardas()

```
bool compareByVardas (
            const Studentas & a,
            const Studentas & b)
```

Compare students by first name.

5.2.1.4 generuotiStudentus()

```
template<typename Container>
void generuotiStudentus (
            Container & grupe,
            int kiekis)
```

Generate random students for testing.

Parameters

grupe	Container to store generated students
kiekis	Number of students to generate

5.2.1.5 isvestiStudentusIFaila()

```
template<typename Container>
void isvestiStudentusIFaila (
            const Container & studentai,
            const std::string & failoPavadinimas,
            char ats)
```

Output students to file.

Parameters

studentai	Container of students
failoPavadinimas	Name of output file
ats	Answer type ('v' - average, 'm' - median)

5.2.1.6 skaitytiIsFailo()

```
template<typename Container>
void skaitytiIsFailo (
            Container & grupe,
            const std::string & failoPavadinimas)
```

Read students from file into container.

Parameters

grupe	Container to store students
failoPavadinimas	Name of the file

5.2.1.7 skirstytiStudentus()

```
template<typename Container>
void skirstytiStudentus (
            Container & grupe,
            Container & kietiakiai,
            Container & vargsai)
```

Separate students into two groups based on final grade.

Parameters

grupe	Original container of students
kietiakiai	Container for students with grade >= 5.0
vargsai	Container for students with grade < 5.0

5.2.1.8 sortStudentai()

```
template<typename Container>
void sortStudentai (
            Container & grupe,
            char sortingOption)
```

Sort students in container.

Parameters

grupe	Container of students
sortingOption	Sorting option ('v' - by name, 'p' - by surname, 'g' - by grade)

5.2.1.9 testuotiDuomenuApdorojima()

```
void testuotiDuomenuApdorojima (
            const std::string & aplankas,
            int skaicius,
            int testChoice)
```

Test data processing performance.

Parameters

aplankas	Directory for files
skaicius	Number of students
testChoice	Test choice (2 for std::vector, 3 for custom Vector)

5.2.1.10 testuotiDuomenuApdorojimaImpl()

```
template<typename Container>
void testuotiDuomenuApdorojimaImpl (
             const std::string & aplankas,
             int skaicius)
```

Implementation of data processing test.

Parameters

aplankas	Directory for test files
skaicius	Number of students

5.2.1.11 testuotiStudentoMetodus()

```
void testuotiStudentoMetodus ()
```

Test Studentas class methods.

5.2.1.12 testuotiZmogausKlase()

```
void testuotiZmogausKlase ()
```

Test Zmogus class.

5.3 funkcijos.h

Go to the documentation of this file.

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include <algorithm>
00008 #include <iomanip>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <stdexcept>
00012 #include <execution>
00013 #include <numeric>
00014 #include <cassert>
00015 #include "zmogus.h"
00016
00024 class Studentas : public Zmogus
00025 {
00026 // realizacija
00027 private:
00028     std::vector<int> nd_;
00029     int egzaminas_;
00030     double galutinis_;
00031
00032 // interfeisas
00033 public:
00035     static int destruktoriuSk;
00036
00041     Studentas() : Zmogus(), egzaminas_(0), galutinis_(0) { }
00042
00047     Studentas(std::istream& is);
00048
```

```
00054     Studentas(const std::string& vardas, const std::string& pavarde)
00055         : Zmogus(vardas, pavarde), egzaminas_(0), galutinis_(0) { }
00056
00057     // Rule of Five
00062     Studentas(const Studentas& other);
00063
00069     Studentas& operator=(const Studentas& other);
00070
00075     Studentas(Studentas&& other) noexcept;
00076
00082     Studentas& operator=(Studentas&& other) noexcept;
00083
00088     ~Studentas() override;
00089
00090     // Implementation of pure virtual methods
00095     void print(std::ostream& os) const override;
00096
00101     void read(std::istream& is) override;
00102
00103     // Getteriai
00108     inline std::vector<int> nd() const { return nd_; }
00109
00114     inline int egzaminas() const { return egzaminas_; }
00115
00120     inline double galutinis() const { return galutinis_; }
00121
00122     // Setteriai
00127     inline void setEgzaminas(int egzaminas) { egzaminas_ = egzaminas; }
00128
00133     inline void setGalutinis(double galutinis) { galutinis_ = galutinis; }
00134
00135     // Metodai
00141     std::istream& readStudent(std::istream& is);
00142
00148     void addND(int pazymys);
00149
00153     void clearND();
00154
00159     double galBalas() const {
00160         if (nd_.empty()) return egzaminas_;
00161         double ndVidurkis = skaiciuotiVid(nd_);
00162         return 0.4 * ndVidurkis + 0.6 * egzaminas_;
00163     }
00164
00169     double galBalasMed() const {
00170         if (nd_.empty()) return egzaminas_;
00171         double ndMediana = skaiciuotiMed(nd_);
00172         return 0.4 * ndMediana + 0.6 * egzaminas_;
00173     }
00174
00180     template <typename Container>
00181     static double skaiciuotiVid(const Container& nd) {
00182         if (nd.empty()) {
00183             throw std::runtime_error("Namu darbu sarasas negali buti tuscias");
00184         }
00185         double suma = 0.0;
00186         for (const auto& elem : nd) {
00187             suma += elem;
00188         }
00189         return suma / nd.size();
00190     }
00191
00197     template <typename Container>
00198     static double skaiciuotiMed(Container nd) {
00199         if (nd.empty()) {
00200             throw std::runtime_error("Namu darbu sarasas negali buti tuscias");
00201         }
00202         std::sort(nd.begin(), nd.end());
00203         size_t dydis = nd.size();
00204         if (dydis % 2 == 0) {
00205             return (nd[dydis / 2 - 1] + nd[dydis / 2]) / 2.0;
00206         } else {
00207             return nd[dydis / 2];
00208         }
00209     }
00210
00216     template<typename Container>
00217 static void nuskaitymasFile(Container& grupe, const std::string& failoPavadinimas) {
00218     std::ifstream failas(failoPavadinimas);
00219     if (!failas.is_open()) {
00220         throw std::runtime_error("Nepavyko atidaryti failo: " + failoPavadinimas);
00221     }
00222
00223     std::string eilute;
00224
00225     // Praleisti pirmą eilutę (header)
00226     if (std::getline(failas, eilute)) {
```

```
00227         // Pirma eilutė praleista
00228     }
00229
00230     // Dabar skaityti studentų duomenis
00231     while (std::getline(failas, eilute)) {
00232         if (eilute.empty()) continue;
00233
00234         std::istringstream iss(eilute);
00235         Studentas studentas;
00236         try {
00237             studentas.readStudent(iss);
00238             studentas.setGalutinis(studentas.galBalas());
00239             grupe.push_back(studentas);
00240         } catch (const std::exception& e) {
00241             std::cerr « "Klaida skaitant studenta: " « e.what() « std::endl;
00242         }
00243     }
00244     failas.close();
00245 }
00246 };
00247
00248 // Comparison functions
00252 bool compareByVardas(const Studentas& a, const Studentas& b);
00253
00257 bool compareByPavarde(const Studentas& a, const Studentas& b);
00258
00262 bool compareByGalutinis(const Studentas& a, const Studentas& b);
00263
00264 // Template functions for container operations
00270 template<typename Container>
00271 void skaitytiIsFailo(Container& grupe, const std::string& failoPavadinimas) {
00272     Studentas::nuskaitymasFile(grupe, failoPavadinimas);
00273 }
00274
00280 template<typename Container>
00281 void sortStudentai(Container& grupe, char sortingOption) {
00282     switch (tolower(sortingOption)) {
00283         case 'v':
00284             std::sort(grupe.begin(), grupe.end(), compareByVardas);
00285             break;
00286         case 'p':
00287             std::sort(grupe.begin(), grupe.end(), compareByPavarde);
00288             break;
00289         case 'g':
00290             std::sort(grupe.begin(), grupe.end(), compareByGalutinis);
00291             break;
00292         default:
00293             std::sort(grupe.begin(), grupe.end(), compareByPavarde);
00294             break;
00295     }
00296 }
00297
00304 template<typename Container>
00305 void skirstytiStudentus(Container& grupe, Container& kietiakiai, Container& vargsai) {
00306     // Using std::partition for efficiency
00307     auto partition_point = std::partition(grupe.begin(), grupe.end(),
00308         [](const Studentas& s) { return s.galutinis() >= 5.0; });
00309
00310     // Copy to respective containers
00311     kietiakiai.assign(grupe.begin(), partition_point);
00312     vargsai.assign(partition_point, grupe.end());
00313
00314     // Clear original container
00315     grupe.clear();
00316 }
00317
00324 template<typename Container>
00325 void isvestiStudentusIFaila(const Container& studentai, const std::string& failoPavadinimas, char ats)
      {
00326     std::ofstream failas(failoPavadinimas);
00327     if (!failas.is_open()) {
00328         throw std::runtime_error("Nepavyko sukurti failo: " + failoPavadinimas);
00329     }
00330
00331     // Header
00332     failas « std::left « std::setw(15) « "Vardas"
00333            « std::setw(15) « "Pavarde"
00334            « std::setw(20) « "Galutinis (" « (tolower(ats) == 'v' ? "Vid." : "Med.") « ")"
00335            « std::endl;
00336     failas « std::string(50, '-') « std::endl;
00337
00338     // Student data
00339     for (const auto& studentas : studentai) {
00340         failas « std::left « std::setw(15) « studentas.vardas()
00341                « std::setw(15) « studentas.pavarde()
00342                « std::fixed « std::setprecision(2)
00343                « (tolower(ats) == 'v' ? studentas.galBalas() : studentas.galBalasMed())
```

```
00344                 « std::endl;
00345     }
00346
00347     failas.close();
00348 }
00349
00355 template<typename Container>
00356 void generuotiStudentus(Container& grupe, int kiekis) {
00357     grupe.clear();
00358     grupe.reserve(kiekis); // Reserve space if container supports it
00359
00360     for (int i = 1; i <= kiekis; ++i) {
00361         Studentas studentas("Vardas" + std::to_string(i), "Pavarde" + std::to_string(i));
00362
00363         // Generate 5-10 random homework grades
00364         int ndKiekis = 5 + rand() % 6;
00365         for (int j = 0; j < ndKiekis; ++j) {
00366             studentas.addND(1 + rand() % 10);
00367         }
00368
00369         // Generate exam grade
00370         studentas.setEgzaminas(1 + rand() % 10);
00371         studentas.setGalutinis(studentas.galBalas());
00372
00373         grupe.push_back(studentas);
00374     }
00375 }
00376
00382 template<typename Container>
00383 void testuotiDuomenuApdorojimaImpl(const std::string& aplankas, int skaicius);
00384
00391 void testuotiDuomenuApdorojima(const std::string& aplankas, int skaicius, int testChoice);
00392
00396 void testuotiStudentoMetodus();
00397
00401 void testuotiZmogausKlase();
00402
00403 #endif // FUNKCIJOS_H
```

5.4 laikas.cpp File Reference

```
#include "Laikas.h"
#include "funkcijos.h"
```

5.5 laikas.h File Reference

```
#include <chrono>
#include <iostream>
```

Classes

- class Laikas

  Class for measuring execution time.

5.6 laikas.h

Go to the documentation of this file.

```
00001 #ifndef LAIKAS_H
00002 #define LAIKAS_H
00003
```

```
00004 #include <chrono>
00005 #include <iostream>
00006
00014 class Laikas
00015 {
00016 private:
00017     std::chrono::high_resolution_clock::time_point start;
00018     std::chrono::high_resolution_clock::time_point end;
00019     std::string veiksmoPavadinimas;
00020
00021 public:
00026     Laikas(const std::string& pavadinimas);
00027
00032     void pradeti();
00033
00038     void baigti();
00039
00044     double gautiLaikoSkirtuma();
00045 };
00046
00047 #endif
```

5.7 main.cpp File Reference

```
#include "funkcijos.h"
#include "laikas.h"
#include "zmogus.h"
#include "vector.h"
```

Functions

- int main ()

5.7.1 Function Documentation

5.7.1.1 main()

```
int main ()
```

5.8 testai.cpp File Reference

```
#include <gtest/gtest.h>
#include "funkcijos.h"
#include "zmogus.h"
```

Classes

- class StudentasTest

Functions

- TEST_F (StudentasTest, DefaultConstructor)
- TEST_F (StudentasTest, CopyConstructor)
- TEST_F (StudentasTest, CopyAssignmentOperator)
- TEST_F (StudentasTest, MoveConstructor)
- TEST_F (StudentasTest, MoveAssignmentOperator)
- TEST_F (StudentasTest, Destructor)
- TEST_F (StudentasTest, GradeCalculation)
- TEST_F (StudentasTest, InputOutput)
- int main (int argc, char **argv)

5.8.1 Function Documentation

5.8.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

5.8.1.2 TEST_F() [1/8]

```
TEST_F (
            StudentasTest ,
            CopyAssignmentOperator )
```

5.8.1.3 TEST_F() [2/8]

```
TEST_F (
            StudentasTest ,
            CopyConstructor )
```

5.8.1.4 TEST_F() [3/8]

```
TEST_F (
            StudentasTest ,
            DefaultConstructor )
```

5.8.1.5 TEST_F() [4/8]

```
TEST_F (
            StudentasTest ,
            Destructor )
```

5.8.1.6 TEST_F() [5/8]

```
TEST_F (
            StudentasTest ,
            GradeCalculation )
```

5.8.1.7 TEST_F() [6/8]

```
TEST_F (
            StudentasTest ,
            InputOutput )
```

5.8.1.8 TEST_F() [7/8]

```
TEST_F (
            StudentasTest ,
            MoveAssignmentOperator )
```

5.8.1.9 TEST_F() [8/8]

```
TEST_F (
            StudentasTest ,
            MoveConstructor )
```

5.9 vector.h File Reference

```
#include <algorithm>
#include <cstddef>
#include <initializer_list>
#include <stdexcept>
#include <utility>
#include <iterator>
#include <memory>
#include <new>
```

Classes

- class Vector< T >

  Dinaminis šabloninis konteineris, panašus į std::vector.

5.10 vector.h

Go to the documentation of this file.

```
00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <algorithm>
00005 #include <cstddef>
00006 #include <initializer_list>
00007 #include <stdexcept>
00008 #include <utility>
00009 #include <iterator>
00010 #include <memory>
00011 #include <new>
00012
00018 template <typename T>
00019 class Vector {
00020 public:
00021     using value_type = T;
00022     using size_type = std::size_t;
00023     using difference_type = std::ptrdiff_t;
00024     using reference = T&;
00025     using const_reference = const T&;
00026     using pointer = T*;
00027     using const_pointer = const T*;
00028     using iterator = T*;
00029     using const_iterator = const T*;
00030     using reverse_iterator = std::reverse_iterator<iterator>;
00031     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00032
00033 private:
00034     pointer data_ = nullptr;
00035     size_type size_ = 0;
00036     size_type capacity_ = 0;
00037     size_t realloc_count_ = 0;
00038
00044     void reallocate(size_type new_capacity) {
00045         if (new_capacity == 0) {
00046             destroy_elements();
00047             std::free(data_);
00048             data_ = nullptr;
00049             size_ = 0;
00050             capacity_ = 0;
00051             return;
00052         }
00053
00054         // Alokuojame naują atmintį
00055         pointer new_data = static_cast<pointer>(std::malloc(new_capacity * sizeof(T)));
00056         if (!new_data) {
00057             throw std::bad_alloc();
00058         }
00059
00060         // Perkeliame esamus elementus
00061         size_type elements_to_move = std::min(size_, new_capacity);
00062         for (size_type i = 0; i < elements_to_move; ++i) {
00063             try {
00064                 new (new_data + i) T(std::move(data_[i]));
00065             } catch (...) {
00066                 // Išvalome jau sukurtus elementus
00067                 for (size_type j = 0; j < i; ++j) {
00068                     (new_data + j)->~T();
00069                 }
00070                 std::free(new_data);
00071                 throw;
00072             }
00073         }
00074
00075         // Išvalome senuosius elementus ir atmintį
00076         destroy_elements();
00077         std::free(data_);
00078
00079         // Nustatome naują būseną
00080         data_ = new_data;
00081         capacity_ = new_capacity;
00082         size_ = elements_to_move;
00083         ++realloc_count_;
00084     }
00085
00089     void destroy_elements() {
00090         for (size_type i = 0; i < size_; ++i) {
00091             (data_ + i)->~T();
00092         }
00093     }
00094
00095 public:
```

```
00099     Vector() = default;
00100
00107     explicit Vector(size_type count, const T& value = T()) {
00108         if (count > 0) {
00109             data_ = static_cast<pointer>(std::malloc(count * sizeof(T)));
00110             if (!data_) {
00111                 throw std::bad_alloc();
00112             }
00113
00114             capacity_ = count;
00115             size_ = 0;
00116
00117             for (size_type i = 0; i < count; ++i) {
00118                 try {
00119                     new (data_ + i) T(value);
00120                     ++size_;
00121                 } catch (...) {
00122                     destroy_elements();
00123                     std::free(data_);
00124                     data_ = nullptr;
00125                     size_ = 0;
00126                     capacity_ = 0;
00127                     throw;
00128                 }
00129             }
00130         }
00131     }
00132
00138     Vector(std::initializer_list<T> init) {
00139         if (init.size() > 0) {
00140             data_ = static_cast<pointer>(std::malloc(init.size() * sizeof(T)));
00141             if (!data_) {
00142                 throw std::bad_alloc();
00143             }
00144
00145             capacity_ = init.size();
00146             size_ = 0;
00147
00148             for (const auto& item : init) {
00149                 try {
00150                     new (data_ + size_) T(item);
00151                     ++size_;
00152                 } catch (...) {
00153                     destroy_elements();
00154                     std::free(data_);
00155                     data_ = nullptr;
00156                     size_ = 0;
00157                     capacity_ = 0;
00158                     throw;
00159                 }
00160             }
00161         }
00162     }
00163
00169     Vector(const Vector& other) {
00170         if (other.size_ > 0) {
00171             data_ = static_cast<pointer>(std::malloc(other.size_ * sizeof(T)));
00172             if (!data_) {
00173                 throw std::bad_alloc();
00174             }
00175
00176             capacity_ = other.size_;
00177             size_ = 0;
00178
00179             for (size_type i = 0; i < other.size_; ++i) {
00180                 try {
00181                     new (data_ + i) T(other.data_[i]);
00182                     ++size_;
00183                 } catch (...) {
00184                     destroy_elements();
00185                     std::free(data_);
00186                     data_ = nullptr;
00187                     size_ = 0;
00188                     capacity_ = 0;
00189                     throw;
00190                 }
00191             }
00192         }
00193     }
00194
00200     Vector(Vector&& other) noexcept
00201         : data_(other.data_), size_(other.size_), capacity_(other.capacity_),
      realloc_count_(other.realloc_count_) {
00202         other.data_ = nullptr;
00203         other.size_ = 0;
00204         other.capacity_ = 0;
00205         other.realloc_count_ = 0;
```

```
00206     }
00207
00211     ~Vector() {
00212         destroy_elements();
00213         std::free(data_);
00214     }
00215
00216     // Operatoriai
00217     Vector& operator=(const Vector& other) {
00218         if (this != &other) {
00219             Vector temp(other);
00220             swap(temp);
00221         }
00222         return *this;
00223     }
00224
00225     Vector& operator=(Vector&& other) noexcept {
00226         if (this != &other) {
00227             destroy_elements();
00228             std::free(data_);
00229
00230             data_ = other.data_;
00231             size_ = other.size_;
00232             capacity_ = other.capacity_;
00233             realloc_count_ = other.realloc_count_;
00234
00235             other.data_ = nullptr;
00236             other.size_ = 0;
00237             other.capacity_ = 0;
00238             other.realloc_count_ = 0;
00239         }
00240         return *this;
00241     }
00242
00243     Vector& operator=(std::initializer_list<T> ilist) {
00244         Vector temp(ilist);
00245         swap(temp);
00246         return *this;
00247     }
00248
00249     // Prieiga prie elementų
00256     reference at(size_type pos) {
00257         if (pos >= size_) {
00258             throw std::out_of_range("Vector::at: index out of range");
00259         }
00260         return data_[pos];
00261     }
00262
00269     const_reference at(size_type pos) const {
00270         if (pos >= size_) {
00271             throw std::out_of_range("Vector::at: index out of range");
00272         }
00273         return data_[pos];
00274     }
00275
00276     reference operator[](size_type pos) { return data_[pos]; }
00277     const_reference operator[](size_type pos) const { return data_[pos]; }
00278     reference front() { return data_[0]; }
00279     const_reference front() const { return data_[0]; }
00280     reference back() { return data_[size_ - 1]; }
00281     const_reference back() const { return data_[size_ - 1]; }
00282     pointer data() noexcept { return data_; }
00283     const_pointer data() const noexcept { return data_; }
00284
00285     // Iteratoriai
00286     iterator begin() noexcept { return data_; }
00287     const_iterator begin() const noexcept { return data_; }
00288     const_iterator cbegin() const noexcept { return data_; }
00289     iterator end() noexcept { return data_ + size_; }
00290     const_iterator end() const noexcept { return data_ + size_; }
00291     const_iterator cend() const noexcept { return data_ + size_; }
00292     reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00293     const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00294     const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00295     reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00296     const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00297     const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00298
00299     // Naudingos funkcijos
00300     bool empty() const noexcept { return size_ == 0; }
00301     size_type size() const noexcept { return size_; }
00302     size_type capacity() const noexcept { return capacity_; }
00303
00304     void reserve(size_type new_cap) {
00305         if (new_cap > capacity_) {
00306             reallocate(new_cap);
00307         }
```

```
00308     }
00309
00310     void shrink_to_fit() {
00311         if (size_ < capacity_) {
00312             reallocate(size_);
00313         }
00314     }
00315
00316     void clear() noexcept {
00317         destroy_elements();
00318         size_ = 0;
00319     }
00320
00321     // Modifikavimas
00322     iterator insert(const_iterator pos, const T& value) {
00323         size_type index = pos - data_;
00324
00325         if (size_ == capacity_) {
00326             size_type new_cap = capacity_ == 0 ? 1 : capacity_ * 2;
00327             reallocate(new_cap);
00328         }
00329
00330         // Pastumti elementus į dešinę
00331         for (size_type i = size_; i > index; --i) {
00332             new (data_ + i) T(std::move(data_[i - 1]));
00333             (data_ + i - 1)->~T();
00334         }
00335
00336         new (data_ + index) T(value);
00337         ++size_;
00338
00339         return data_ + index;
00340     }
00341
00342     iterator insert(const_iterator pos, T&& value) {
00343         size_type index = pos - data_;
00344
00345         if (size_ == capacity_) {
00346             size_type new_cap = capacity_ == 0 ? 1 : capacity_ * 2;
00347             reallocate(new_cap);
00348         }
00349
00350         // Pastumti elementus į dešinę
00351         for (size_type i = size_; i > index; --i) {
00352             new (data_ + i) T(std::move(data_[i - 1]));
00353             (data_ + i - 1)->~T();
00354         }
00355
00356         new (data_ + index) T(std::move(value));
00357         ++size_;
00358
00359         return data_ + index;
00360     }
00361
00362     iterator insert(const_iterator pos, size_type count, const T& value) {
00363         if (count == 0) return const_cast<iterator>(pos);
00364
00365         size_type index = pos - data_;
00366
00367         if (size_ + count > capacity_) {
00368             size_type new_cap = std::max(capacity_ * 2, size_ + count);
00369             reallocate(new_cap);
00370         }
00371
00372         // Pastumti esamus elementus į dešinę
00373         for (size_type i = size_ + count - 1; i >= index + count && i < size_ + count; --i) {
00374             new (data_ + i) T(std::move(data_[i - count]));
00375             (data_ + i - count)->~T();
00376         }
00377
00378         // Įdėti naujus elementus
00379         for (size_type i = 0; i < count; ++i) {
00380             new (data_ + index + i) T(value);
00381         }
00382
00383         size_ += count;
00384         return data_ + index;
00385     }
00386
00387     iterator erase(const_iterator pos) {
00388         size_type index = pos - data_;
00389
00390         (data_ + index)->~T();
00391
00392         // Pastumti elementus į kairę
00393         for (size_type i = index; i < size_ - 1; ++i) {
00394             new (data_ + i) T(std::move(data_[i + 1]));
```

```
00395             (data_ + i + 1)->~T();
00396         }
00397
00398         --size_;
00399         return data_ + index;
00400     }
00401
00402     iterator erase(const_iterator first, const_iterator last) {
00403         size_type start_index = first - data_;
00404         size_type end_index = last - data_;
00405         size_type count = end_index - start_index;
00406
00407         if (count == 0) return const_cast<iterator>(first);
00408
00409         // Sunaikinti elementus intervale
00410         for (size_type i = start_index; i < end_index; ++i) {
00411             (data_ + i)->~T();
00412         }
00413
00414         // Pastumti likusius elementus į kairę
00415         for (size_type i = start_index; i < size_ - count; ++i) {
00416             new (data_ + i) T(std::move(data_[i + count]));
00417             (data_ + i + count)->~T();
00418         }
00419
00420         size_ -= count;
00421         return data_ + start_index;
00422     }
00423
00424     void push_back(const T& value) {
00425         if (size_ == capacity_) {
00426             size_type new_cap = capacity_ == 0 ? 1 : capacity_ * 2;
00427             reallocate(new_cap);
00428         }
00429
00430         new (data_ + size_) T(value);
00431         ++size_;
00432     }
00433
00434     void push_back(T&& value) {
00435         if (size_ == capacity_) {
00436             size_type new_cap = capacity_ == 0 ? 1 : capacity_ * 2;
00437             reallocate(new_cap);
00438         }
00439
00440         new (data_ + size_) T(std::move(value));
00441         ++size_;
00442     }
00443
00451     template <class... Args>
00452     reference emplace_back(Args&&... args) {
00453         if (size_ == capacity_) {
00454             size_type new_cap = capacity_ == 0 ? 1 : capacity_ * 2;
00455             reallocate(new_cap);
00456         }
00457
00458         new (data_ + size_) T(std::forward<Args>(args)...);
00459         ++size_;
00460         return data_[size_ - 1];
00461     }
00462
00463     void pop_back() {
00464         if (size_ > 0) {
00465             --size_;
00466             (data_ + size_)->~T();
00467         }
00468     }
00469
00470     void resize(size_type count) {
00471         resize(count, T());
00472     }
00473
00474     void resize(size_type count, const T& value) {
00475         if (count < size_) {
00476             // Sumažinti dydį
00477             for (size_type i = count; i < size_; ++i) {
00478                 (data_ + i)->~T();
00479             }
00480             size_ = count;
00481         } else if (count > size_) {
00482             // Padidinti dydį
00483             if (count > capacity_) {
00484                 reallocate(count);
00485             }
00486
00487             for (size_type i = size_; i < count; ++i) {
00488                 new (data_ + i) T(value);
```

```
00489             }
00490             size_ = count;
00491         }
00492     }
00493
00494     void swap(Vector& other) noexcept {
00495         std::swap(data_, other.data_);
00496         std::swap(size_, other.size_);
00497         std::swap(capacity_, other.capacity_);
00498         std::swap(realloc_count_, other.realloc_count_);
00499     }
00500
00501     // Naujos funkcijos
00502     void assign(size_type count, const T& value) {
00503         clear();
00504         if (count > capacity_) {
00505             reallocate(count);
00506         }
00507
00508         for (size_type i = 0; i < count; ++i) {
00509             new (data_ + i) T(value);
00510         }
00511         size_ = count;
00512     }
00513
00514     template<class InputIt>
00515     void assign(InputIt first, InputIt last) {
00516         clear();
00517         for (auto it = first; it != last; ++it) {
00518             push_back(*it);
00519         }
00520     }
00521
00522     void assign(std::initializer_list<T> ilist) {
00523         clear();
00524         if (ilist.size() > capacity_) {
00525             reallocate(ilist.size());
00526         }
00527
00528         size_type i = 0;
00529         for (const auto& item : ilist) {
00530             new (data_ + i) T(item);
00531             ++i;
00532         }
00533         size_ = ilist.size();
00534     }
00535
00541     size_t reallocations() const {
00542         return realloc_count_;
00543     }
00544
00545     // Lyginimo operatoriai
00546     friend bool operator==(const Vector& lhs, const Vector& rhs) {
00547         return lhs.size_ == rhs.size_ && std::equal(lhs.data_, lhs.data_ + lhs.size_, rhs.data_);
00548     }
00549
00550     friend bool operator!=(const Vector& lhs, const Vector& rhs) {
00551         return !(lhs == rhs);
00552     }
00553 };
00554
00555 #endif // VECTOR_H
```

5.11 vector_testai.cpp File Reference

```
#include <gtest/gtest.h>
#include "vector.h"
#include <vector>
#include <string>
#include <algorithm>
#include <stdexcept>
```

Classes

- class VectorTest

Functions

- TEST_F (VectorTest, DefaultConstructor)
- TEST_F (VectorTest, CountValueConstructor)
- TEST_F (VectorTest, InitializerListConstructor)
- TEST_F (VectorTest, CopyConstructor)
- TEST_F (VectorTest, MoveConstructor)
- TEST_F (VectorTest, CopyAssignment)
- TEST_F (VectorTest, MoveAssignment)
- TEST_F (VectorTest, InitializerListAssignment)
- TEST_F (VectorTest, ElementAccess)
- TEST_F (VectorTest, CapacityOperations)
- TEST_F (VectorTest, PushBackAndPopBack)
- TEST_F (VectorTest, PushBackMoveSemantics)
- TEST_F (VectorTest, EmplaceBack)
- TEST_F (VectorTest, Clear)
- TEST_F (VectorTest, Resize)
- TEST_F (VectorTest, InsertSingleElement)
- TEST_F (VectorTest, InsertMoveElement)
- TEST_F (VectorTest, InsertMultipleElements)
- TEST_F (VectorTest, EraseSingleElement)
- TEST_F (VectorTest, EraseRange)
- TEST_F (VectorTest, Iterators)
- TEST_F (VectorTest, ComparisonOperators)
- TEST_F (VectorTest, AlgorithmCompatibility)
- TEST_F (VectorTest, EdgeCases)
- TEST_F (VectorTest, ReserveEfficiency)
- int main (int argc, char **argv)

5.11.1 Function Documentation

5.11.1.1 main()

```
int main (
            int argc,
            char ** argv)
```

5.11.1.2 TEST_F() [1/25]

```
TEST_F (
            VectorTest ,
            AlgorithmCompatibility )
```

5.11.1.3 TEST_F() [2/25]

```
TEST_F (
            VectorTest ,
            CapacityOperations )
```

5.11.1.4 TEST_F() [3/25]

```
TEST_F (
            VectorTest ,
            Clear )
```

5.11.1.5 TEST_F() [4/25]

```
TEST_F (
            VectorTest ,
            ComparisonOperators )
```

5.11.1.6 TEST_F() [5/25]

```
TEST_F (
            VectorTest ,
            CopyAssignment )
```

5.11.1.7 TEST_F() [6/25]

```
TEST_F (
            VectorTest ,
            CopyConstructor )
```

5.11.1.8 TEST_F() [7/25]

```
TEST_F (
            VectorTest ,
            CountValueConstructor )
```

5.11.1.9 TEST_F() [8/25]

```
TEST_F (
            VectorTest ,
            DefaultConstructor )
```

5.11.1.10 TEST_F() [9/25]

```
TEST_F (
            VectorTest ,
            EdgeCases )
```

5.11.1.11 TEST_F() [10/25]

```
TEST_F (
            VectorTest ,
            ElementAccess )
```

5.11.1.12 TEST_F() [11/25]

```
TEST_F (
            VectorTest ,
            EmplaceBack )
```

5.11.1.13 TEST_F() [12/25]

```
TEST_F (
            VectorTest ,
            EraseRange )
```

5.11.1.14 TEST_F() [13/25]

```
TEST_F (
            VectorTest ,
            EraseSingleElement )
```

5.11.1.15 TEST_F() [14/25]

```
TEST_F (
            VectorTest ,
            InitializerListAssignment )
```

5.11.1.16 TEST_F() [15/25]

```
TEST_F (
            VectorTest ,
            InitializerListConstructor )
```

5.11.1.17 TEST_F() [16/25]

```
TEST_F (
            VectorTest ,
            InsertMoveElement )
```

5.11.1.18 TEST_F() [17/25]

```
TEST_F (
            VectorTest ,
            InsertMultipleElements )
```

5.11.1.19 TEST_F() [18/25]

```
TEST_F (
            VectorTest ,
            InsertSingleElement )
```

5.11.1.20 TEST_F() [19/25]

```
TEST_F (
            VectorTest ,
            Iterators )
```

5.11.1.21 TEST_F() [20/25]

```
TEST_F (
            VectorTest ,
            MoveAssignment )
```

5.11.1.22 TEST_F() [21/25]

```
TEST_F (
            VectorTest ,
            MoveConstructor )
```

5.11.1.23 TEST_F() [22/25]

```
TEST_F (
            VectorTest ,
            PushBackAndPopBack )
```

5.11.1.24 TEST_F() [23/25]

```
TEST_F (
            VectorTest ,
            PushBackMoveSemantics )
```

5.11.1.25 TEST_F() [24/25]

```
TEST_F (
            VectorTest ,
            ReserveEfficiency )
```

5.11.1.26 TEST_F() [25/25]

```
TEST_F (
            VectorTest ,
            Resize )
```

5.12 zmogus.cpp File Reference

```
#include "zmogus.h"
```

Functions

- std::ostream & operator<< (std::ostream &os, const Zmogus &zmogus)

  Output operator for Zmogus class.
- std::istream & operator>> (std::istream &is, Zmogus &zmogus)

  Input operator for Zmogus class.

5.12.1 Function Documentation

5.12.1.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & os,
            const Zmogus & zmogus)
```

Output operator for Zmogus class.

Parameters

os	Output stream
zmogus	Person to output

Returns

Reference to output stream

5.12.1.2 operator>>()

```
std::istream & operator>> (
            std::istream & is,
            Zmogus & zmogus)
```

Input operator for Zmogus class.

Parameters

is	Input stream
zmogus	Person to input to

Returns

Reference to input stream

5.13 zmogus.h File Reference

```
#include <string>
#include <iostream>
```

Classes

- class Zmogus

  Abstract base class representing a person.

Functions

- std::ostream & operator<< (std::ostream &os, const Zmogus &zmogus)

  Output operator for Zmogus class.
- std::istream & operator>> (std::istream &is, Zmogus &zmogus)

  Input operator for Zmogus class.

5.13.1 Function Documentation

5.13.1.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & os,
            const Zmogus & zmogus)
```

Output operator for Zmogus class.

Parameters

os	Output stream
zmogus	Person to output

Returns

Reference to output stream

5.13.1.2 operator>>()

```
std::istream & operator>> (
            std::istream & is,
            Zmogus & zmogus)
```

Input operator for Zmogus class.

Parameters

is	Input stream
zmogus	Person to input to

Returns

Reference to input stream

5.14 zmogus.h

Go to the documentation of this file.

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00014 class Zmogus
00015 {
00016 protected:
00017     std::string vardas_;
00018     std::string pavarde_;
00019
00020 public:
00024     Zmogus() = default;
00025
00031     Zmogus(const std::string& vardas, const std::string& pavarde)
00032         : vardas_(vardas), pavarde_(pavarde) {}
00033
00034     // Rule of Five
00039     Zmogus(const Zmogus& other) : vardas_(other.vardas_), pavarde_(other.pavarde_) {}
00040
00046     Zmogus& operator=(const Zmogus& other);
00047
00052     Zmogus(Zmogus&& other) noexcept;
00053
00059     Zmogus& operator=(Zmogus&& other) noexcept;
00060
00065     virtual ~Zmogus() = 0;
00066
00073     virtual void print(std::ostream& os) const = 0;
00074
00081     virtual void read(std::istream& is) = 0;
00082
00083     // Getters and setters
00088     inline std::string vardas() const { return vardas_; }
00089
00094     inline std::string pavarde() const { return pavarde_; }
00095
00100     inline void setVardas(const std::string& vardas) { vardas_ = vardas; }
00101
00106     inline void setPavarde(const std::string& pavarde) { pavarde_ = pavarde; }
00107 };
00108
00115 std::ostream& operator<<(std::ostream& os, const Zmogus& zmogus);
00116
00123 std::istream& operator>>(std::istream& is, Zmogus& zmogus);
00124
00125 #endif // ZMOGUS_H
```

Index